

14장 포인터와 함수에 대한 이해

1. call by value vs. call by reference
2. 함수의 인자로 배열 전달하기
3. 포인터 대상의 const 선언



지역 변수(local variable)

3

- 함수 안에서 선언된 변수

```
int compute_sum(int n)
{
    int i, result = 0;
    for(i=0; i<=n; i++)
        result += i;
    return result;
}
```

지역 변수



지역 변수의 사용 범위

4

- 지역 변수는 선언된 함수 안에서만 사용(접근) 가능
- 다른 함수에서 사용시 컴파일 에러 발생

```
int sub1()
```

```
{  
  
  
  
  
  
}
```

```
    int x;
```

```
    x=10;
```

```
    ...
```

지역 변수 x의 유효한 범위

```
void sub2()
```

```
{
```

```
    printf("%d \n", x);
```

```
}
```

// 다른 함수의 지역변수 접근 불가능
// 컴파일시 에러 발생!!!



지역 변수의 생성 기간

5

- 함수가 호출되면 메모리에 생성되고 종료되면 자동으로 소멸 -> 함수가 실행되는 동안만 메모리에 존재

```
int sub()
{
    int i=0,result;           // 지역변수 생성

    ...

    return result;           // 지역변수 소멸
}
```



지역 변수의 초기값

6

- 지역변수는 초기화 하지 않으면 쓰레기값을 가진다.
- 쓰레기값 -> 이전에 같은 메모리공간을 사용한 프로그램이 저장한 값

```
#include <stdio.h>
int main(void)
{
    int temp;                // 초기화하지 않으면 쓰레기값
    printf("temp=%d\n",temp);
    return 0;
}
```

temp=-2523352



함수의 매개 변수

7

- 매개 변수도 일종의 지역 변수
- 함수를 호출할 때 ① 매개변수가 메모리에 할당되고 ②인자값으로 초기화

```
int inc(int counter) // 매개변수 할당 및 인자로 초기화
```

```
{  
    counter++;  
    return counter;  
}
```

```
int main(void)
```

```
{  
    ...  
    result = inc(value);  
    ...  
}
```

```
int inc()
```

```
{
```

```
int counter = value;
```

```
counter++;
```

```
return counter;
```

```
}
```



같은 이름의 지역 변수

8

- 같은 이름의 지역 변수가 존재해도 선언된 함수가 다르다면 문제없음

```
int sub1()
{
    int x=0;
    ...
}
void sub2()
{
    int x=0;
    ...
}
void sub3(int x)
{
    x=0;
    ...
}
```

// 이름이 같아도 다른 변수임



함수사이에 데이터 전달방식

9

- 다른 함수에서 선언된 지역변수를 사용(직접 참조)할 수 없음
- 함수안에서 함수 밖을 볼 수 없고 함수 밖에서 안을 볼 수도 없음

```
#include <stdio.h>
void print(void);           // 함수선언
int main(void)
{
    int number = 1;
    print();                 // 함수호출
    return 0;
}
void print(void)             // 함수정의
{
    printf("값은 %d입니다.\n", number); // 컴파일 에러 발생
}
```



함수사이에 데이터 전달방법

10

- 다른 함수(함수 외부)에서 선언된 지역변수를 사용하려면 함수의 매개변수를 통하여 데이터를 전달받아서 사용해야 함
- 처리 결과를 함수 외부로 전달하려면 반환값을 통하여 전달함

```
#include <stdio.h>
void print(int number);
int main(void)
{
    int number = 1;
    print(number);
    return 0;
}
void print(int value)
{
    printf("값은 %d입니다.\n", value);
}
```





함수사이에 데이터 전달 방법

11

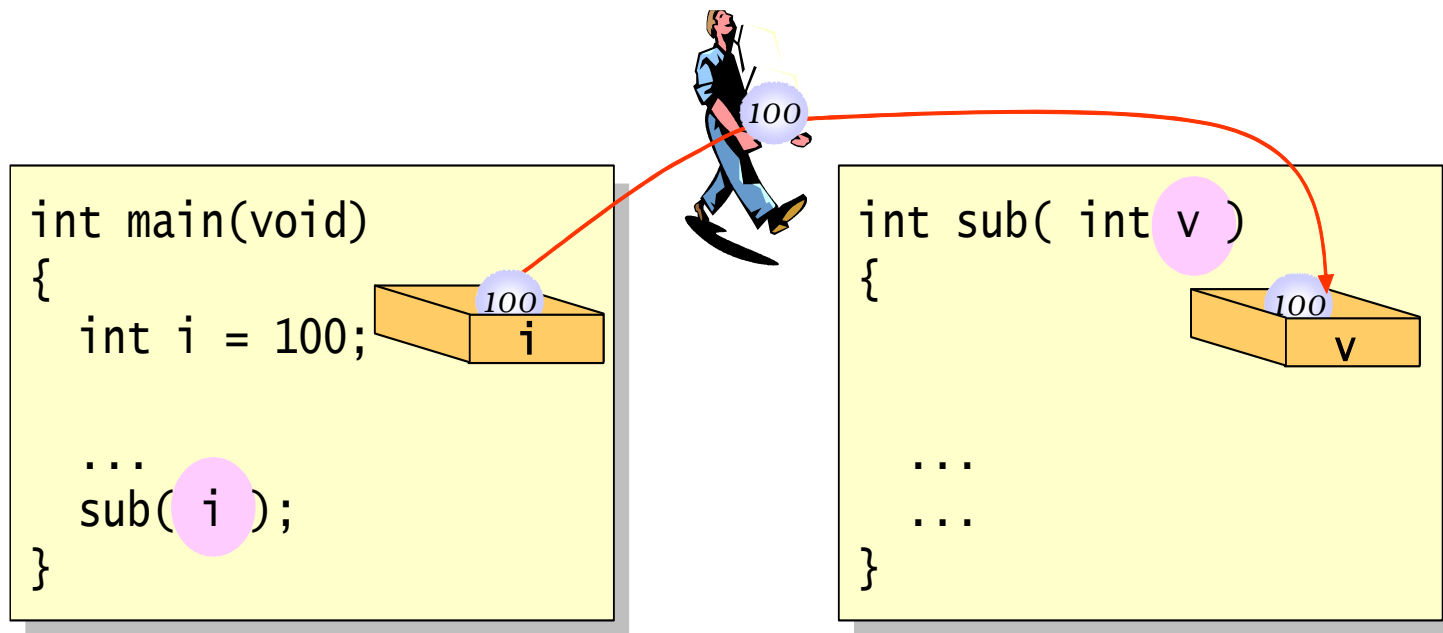
- 값에 의한 호출(call by value)
 - ▣ 변수의 값을 전달하는 방식, C의 기본적인 호출방법
 - ▣ 매개변수를 일반 변수로 선언하고 함수의 매개변수를 통하여 값을 전달
 - ▣ 복사본이 전달된다고 생각하면 됨
- 주소에 의한 호출(call by pointer)
 - ▣ 변수의 주소를 전달하는 방식
 - ▣ 매개변수를 포인터로 선언하고 함수의 매개변수에 주소를 전달
 - ▣ 원본이 전달된다고 생각하면 됨
- 참조에 의한 호출(call by reference)
 - ▣ C언어에서는 지원 안됨, 객체지향형 언어에서 지원함(C++, Python)
 - ▣ 주소에 의한 호출과 동일한 결과를 얻을 수 있음



값에 의한 호출

12

- 함수를 호출할 때 인자에 변수의 값을 전달
- 함수의 매개변수는 변수의 값을 받을 수 있는 일반변수로 선언



함수 호출시 자동으로 실행되는 2가지

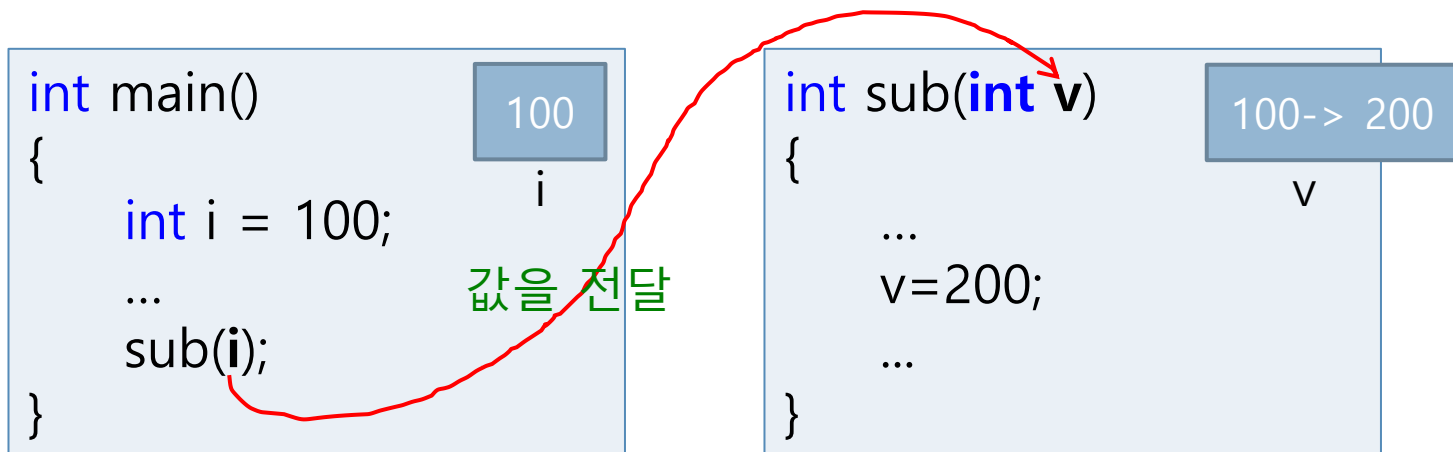
- ① 매개변수를 메모리에 할당
- ② 매개변수를 인자로 초기화



인자와 매개변수의 관계

13

- 인자는 호출하는 함수 쪽에서 선언된 변수이고 매개변수는 호출되는 함수 쪽에서 선언된 변수임
- 함수가 호출되는 순간 인자의 값이 매개변수에 전달됨
- 인자와 매개변수는 별개의 변수임
- 매개변수를 변경하더라도 인자가 변경되지는 않음

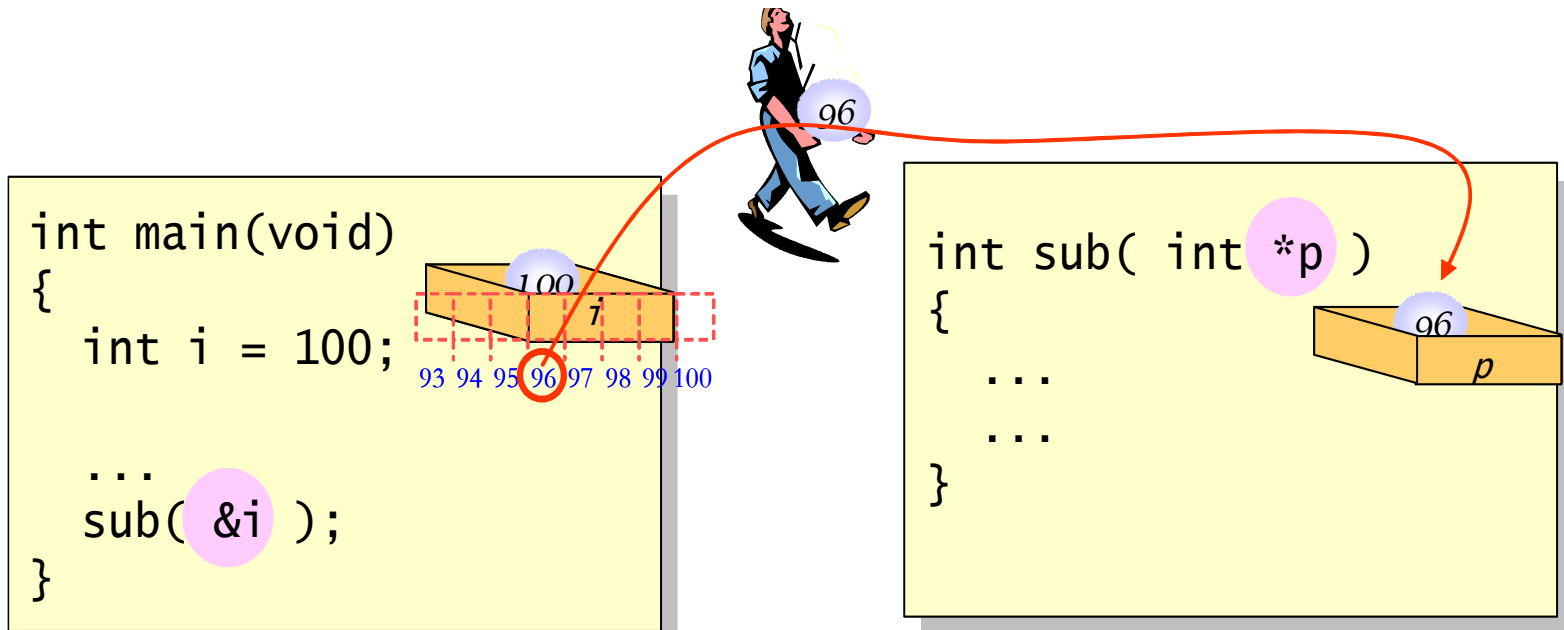




주소에 의한 호출

14

- 함수를 호출할 때 인자에 변수의 주소를 전달
- 함수의 매개변수는 주소를 받을 수 있도록 포인터로 선언



함수 호출시 자동으로 실행되는 2가지
① 매개변수를 메모리에 할당
② 매개변수를 인자로 초기화

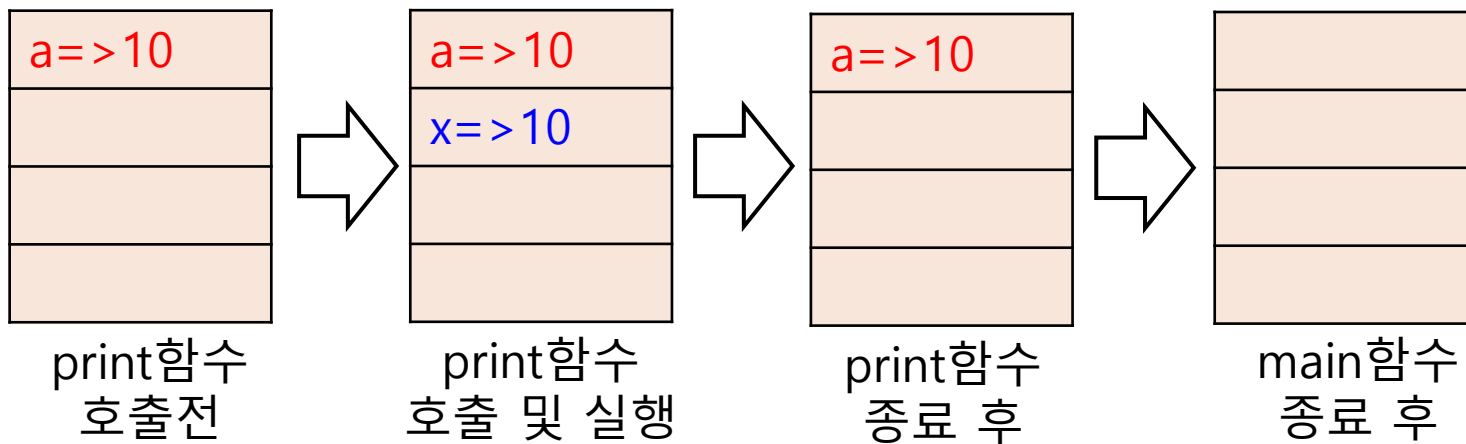


값에 의한 호출 예제

15

```
#include <stdio.h>
void print(int x);
int main(void)
{
    int a = 10;
    print(a);           // 값에 의한 호출
    return 0;
}
int print(int x)        // 호출시 매개변수 할당 & 인자로 초기화
{
    printf("x의 값은 %d입니다.\n", x);
}
```

x의 값은 10입니다.





값에 의한 호출의 문제점

16

- 값에 의한 호출방식으로 다른 함수에서 선언된 지역변수의 값을 변경하는 것은 불가능

```
#include <stdio.h>
void modify(int value);
int main(void)
{
    int number = 1;
    modify(number);
    printf("number = %d\n", number);

    return 0;
}
void modify(int value) // main함수의 number를 99로 변경하는 함수
{
    value = 99;
}
```

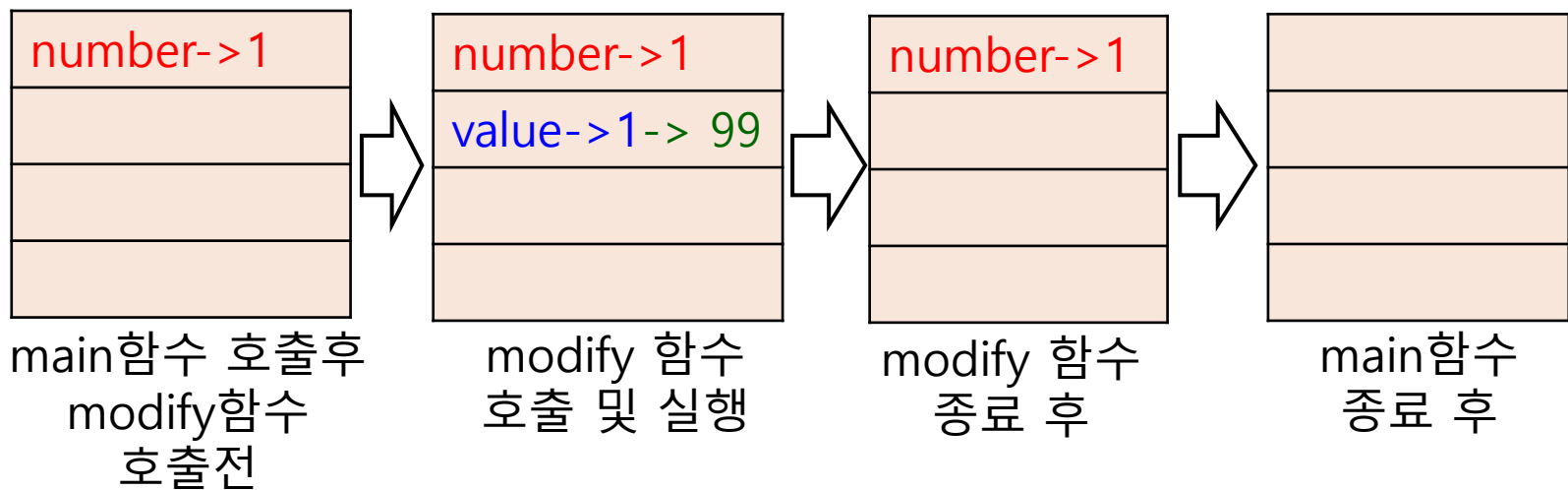
number = 1



값에 의한 호출의 문제점

17

- 시간 순서에 따른 메모리 변화





주소에 의한 호출

18

- 주소에 의한 호출방식은 다른 함수에서 선언된 지역변수의 값을 변경하는 것은 가능-> 주소로 다른 함수의 변수 접근(간접 참조)은 가능

```
#include <stdio.h>
void modify(int* ptr);
int main(void)
{
    int number = 1;
    modify(&number);
    printf("number = %d\n", number);

    return 0;
}
void modify(int* ptr)
{
    *ptr = 99;          // *500 = 99 -> main/number = 99
}
```

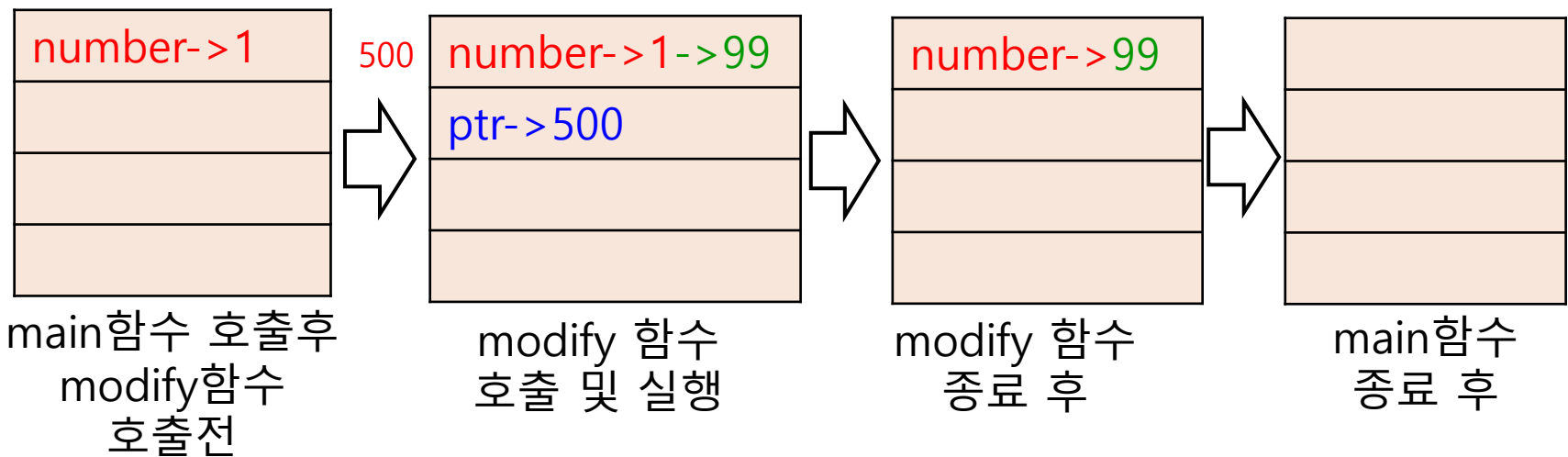
number = 99



주소에 의한 호출

19

- 시간 순서에 따른 메모리 변화





swap 예제: 함수 미사용

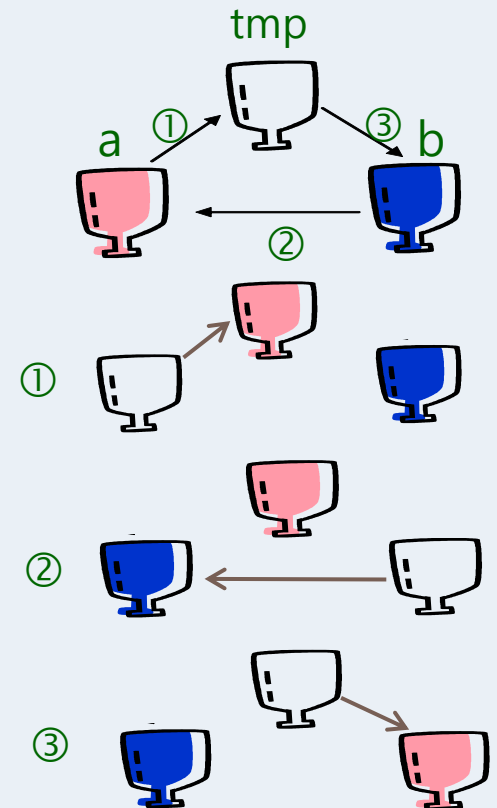
20

```
#include <stdio.h>
int main(void)
{
    int a = 100, b = 200, tmp;
    printf("a=%d b=%d\n", a, b);

    tmp = a;      //①
    a = b;        //②
    b = tmp;      //③

    printf("a=%d b=%d\n", a, b);
    return 0;
}
```

a=100 b=200
a=200 b=100





swap 함수: 값에 의한 호출

21

```
#include <stdio.h>
void swap(int x, int y);
int main(void)
{
    int a = 100, b = 200;
    printf("a=%d b=%d\n", a, b);
    swap(a, b);
    printf("a=%d b=%d\n", a, b);
    return 0;
}

void swap(int x, int y) //값에 의한 호출
{
    int tmp;
    tmp = x;
    x = y;
    y = tmp;
}
```

a=100 b=200
a=100 b=200

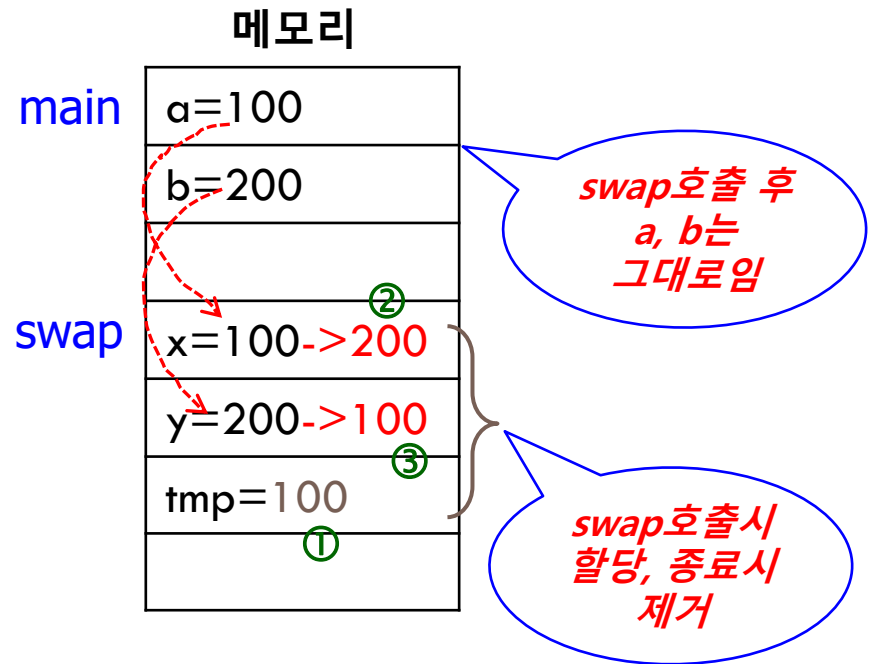
변경되지 않음!!
Why?



swap 함수: 값에 의한 호출

22

```
#include <stdio.h>
void swap(int x, int y);
int main(void)
{
    int a = 100, b = 200;
    printf("a=%d b=%d\n", a, b);
    swap(a, b);
    printf("a=%d b=%d\n", a, b);
    return 0;
}
void swap(int x, int y)
{
    int tmp;
    tmp = x;      //①
    x = y;        //②
    y = tmp;      //③
}
```



- 메인함수의 a, b는 바뀌지 않고 매개변수 x, y의 값들만 교환됨
- 값에 의한 호출방식으로는 다른 함수의 지역변수들을 변경할 수 없음



swap 함수: 주소에 의한 호출

23

```
#include <stdio.h>
void swap(int* px, int* py);
int main(void)
{
    int a = 100, b = 200;
    printf("a=%d b=%d\n", a, b);
    swap(&a, &b);
    printf("a=%d b=%d\n", a, b);
    return 0;
}
void swap(int* px, int* py) //주소에 의한 호출
{
    int tmp;
    tmp = *px;    //①
    *px = *py;    //②
    *py = tmp;    //③
}
```

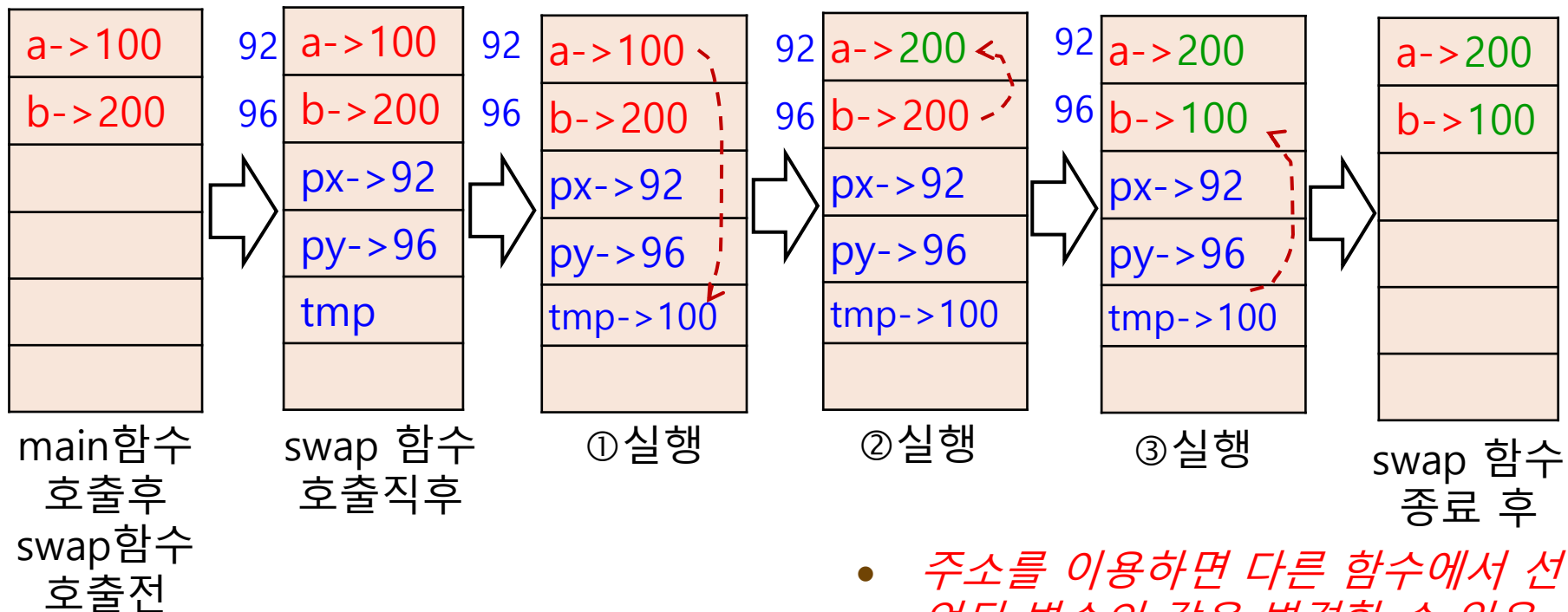
a=100 b=200
a=200 b=100



swap 함수: 주소에 의한 호출

24

□ 시간 순서에 따른 메모리 변화



- 주소를 이용하면 다른 함수에서 선언된 변수의 값을 변경할 수 있음

- 값에 의한 호출 방식은 다른 함수에서 선언된 변수의 값을 변경할 수 없음.
- 다른 함수에서 선언된 지역 변수의 값을 변경하고자 하는 경우 반드시 주소에 의한 호출 방법을 사용해야 함 -> 포인터 변수가 필요한 첫번째 이유임



scanf 함수 &의 비밀

26

- scanf 함수 사용시 변수 앞에 &를 붙이는 이유
 - ▣ scanf 함수의 두번째 매개변수는 포인터로 선언되어 있음(stdio.h)
 - ▣ scanf 함수 호출시 인자로 주소를 전달해야 함
 - ▣ scanf 함수는 다른 함수에서 선언된 변수에 키보드로 부터 받은 값을 저장(변경)해줘야 하므로 주소에 의한 호출을 사용

```
#include <stdio.h>
int main(void)
{
    int a;
    printf("정수 입력하시오:");
    scanf("%d", &a);        //다른 함수(main)의 변수a를 변경함
    ....
}
```

scanf 함수 &의 비밀

27

- 배열명은 주소이므로 &를 붙이지 않음

```
#include <stdio.h>
int main(void)
{
    char str[30];
    printf("문자열을 입력하시오: ");
    scanf("%s", str);          //str은 배열의 시작주소
    ....
}
```



실습과제 1

28

- 함수의 인자와 매개변수의 차이를 설명하라.
- 함수가 호출될 때 컴퓨터에 의해 자동으로 실행되는 2가지를 설명하라.
- 값에 의한 함수 호출방식의 문제점을 설명하라.
- 주소에 의한 함수 호출방식이 필요한 경우를 설명하라.



실습과제2

29

- 아래 코드에서 add2함수는 매개변수의 값을 2만큼 증가시키는 함수이다. 그러나 실행결과에서 보듯이 원하는 대로 동작하지 않는다. 그 이유를 17페이지 메모리 그림을 이용하여 자세히 설명하고 원하는 결과가 나오도록 코드를 수정하시오.

```
#include <stdio.h>
void add2(int value);
int main(void)
{
    int number;
    printf("정수를 입력하세요:");
    scanf("%d", &number);
    add2(number);
    printf("2만큼 증가한 값:%d\n", number);
    return 0;
}
void add2(int value)
{
    value += 2;
}
```

정수를 입력 하시오: 15 <엔터>
2만큼 증가한 값: 15



실습과제3

30

- 과제2번의 함수를 값에 의한 호출을 이용하여 2를 증가시킨 후 리턴하고 main함수에서 리턴값을 받아서 처리하는 방식으로 다음처럼 실행되도록 하시오. 반드시 함수의 선언, 호출, 정의를 모두 사용할 것

정수를 입력 하시오: 15 <엔터>
2만큼 증가한 값: 17



실습과제 4

31

- swap함수 예제를 참고하여 아래 결과가 나오도록 코드를 수정 하시오.
세 정수를 교환하는 부분은 반드시 함수(swap)로 작성하고 또, 함수의 선언, 호출, 정의를 모두 사용할 것
- 세정수의 교환방법은 실행결과로부터 유추하라.
- 24페이지처럼 시간에 따라 메모리의 상태를 그려보시오.

```
정수x를 입력 하시오: 10 <엔터>  
정수y를 입력 하시오: 20 <엔터>  
정수z를 입력 하시오: 30 <엔터>  
swap함수 호출 전 x=10, y=20, z=30  
swap함수 호출 후 x=20, y=30, z=10
```

- 교재 320페이지 문제1번을 푸시오. 단, num 변수에 저장된 값을 100배 하는 함수로 수정하라.
- 함수의 선언, 호출, 정의를 모두 사용할 것
- 2가지 방식의 차이를 설명하고 원하는 결과는 어느 것인가?



과제제출방법

33

- 아래 주소의 작성방법대로 github에 제출할 것
 - ▣ <https://github.com/2sungryul/cprogramming/tree/main>
- 기한 : 이클래스 과제 게시판의 마감시간 참조
- 소스코드의 첫 부분은 아래처럼 제목,날짜,작성자(학번,이름)를 작성할 것

```
// *****  
//   제   목   : 정수 4개의 평균을 구하는 프로그램  
//   날   짜   : 2023년 9월10일  
//   작성자   : 15010101 홍길동  
// *****  
  
// 소스코드 작성
```